const_missing

Remember our discussion of Rake in *The Rake Example*, on page 23? In that section we said that at one point in its history, Rake renamed classes like Task to names that are less likely to clash, such as Rake::Task. After renaming the classes, Rake went through an upgrade path: for a few versions, you could use either the new class names or the old, non-Namespaced names. Rake allowed you to do that by *Monkepatching (16)* the Module#const_missing method:

gems/rake-0.9.2.2/lib/rake/ext/module.rb

```
class Module
  def const_missing(const_name)
    case const_name
    when :Task
      Rake.application.const_warning(const_name)
      Rake::Task
    when :FileTask
      Rake.application.const_warning(const_name)
      Rake::FileTask
    when :FileCreationTask
      # ...
    end
  end
end
```

When you reference a constant that doesn't exist, Ruby passes the name of the constant to const_missing as a symbol. Class names are just constants, so a reference to an unknown Rake class such as Task was routed to Module#const_missing. In turn, const_missing warned you that you were using an obsolete class name:

methods/const_missing.rb

```
require 'rake'
task_class = Task
```

```
WARNING: Deprecated reference to top-level constant 'Task' found [...]
    Use --classic-namespace on rake command
    or 'require "rake/classic_namespace"' in Rakefile
```

After the warning, you automatically got the new, Namespaced class name in place of the old one:

```
task_class        # => Rake::Task
```

Enough talking about magic methods. Let's recap what you and Bill did today.

Refactoring Wrap-Up

Today you solved the same problem in two different ways. The first version of Computer introspects DS to get a list of methods to wrap and uses *Dynamic*